

VS Code 联合 Vivado 开发指南

由于Vivado的编辑器和记事本写代码没什么区别，将VS Code与Vivado联合使用是目前FPGA和数字电路开发的主流方案。你可以通过将VS Code设置为Vivado的外部编辑器，并在VS Code中配置对应的语言插件和语法检查器来彻底替代Vivado自带的编辑器。

以下是严谨且完整的配置步骤：

1. 将VS Code设置为Vivado的默认编辑器

这使得你在Vivado中双击源文件时，系统会自动唤起VS Code并定位到对应的代码行。

1. 打开Vivado，在顶部菜单栏选择 **Tools -> Settings**。
2. 在弹出的Settings窗口左侧，展开 **Tool Settings**，点击 **Text Editor**。
3. 在右侧的 **Current Editor** 下拉菜单中，将默认的 **Vivado Text Editor** 更改为 **Custom Editor**。
4. 弹出来的窗口中，在**Editor**的空格中输入以下命令（注意保留括号和内部的空格，这是Vivado的环境变量占位符）：

如果你的VS Code已经添加到了系统环境变量（推荐）：

```
code -g [file name]:[line number]
```

如果没有添加到系统环境变量，请使用绝对路径（路径请根据实际安装位置替换，建议使用正斜杠）：

- **User Installer（仅当前用户，最常见）：** `"C:/Users/<你的实际用户名>/AppData/Local/Programs/Microsoft VS Code/bin/code.cmd" -g [file name]:[line number]`（注意：AppData 是隐藏文件夹，如果找不到，可以直接在文件资源管理器地址栏输入该路径）
- **System Installer（为所有用户安装）：** `"C:/Program Files/Microsoft VS Code/bin/code.cmd" -g [file name]:[line number]`

点击 **OK** 保存。

2. 配置VS Code的Verilog/SystemVerilog开发环境

Vivado自带编辑器的糟糕体验主要来源于缺乏现代IDE的自动补全、格式化和实时语法检查。你需要在VS Code中安装以下核心插件：

- **Verilog-HDL/SystemVerilog/Bluespec SystemVerilog（作者: mshr-h）：**
这是目前使用最广泛的Verilog基础支持插件。提供代码高亮、代码片段补全以及集成第三方Linter（语法检查器）的功能。
- **TerosHDL（可选，但强烈推荐）：**
这是一个重型插件，提供类似于专业IDE的功能，包括状态机视图、模块层级视图、实例化模板自动生成等，非常适合大型工程管理。

3. 配置基于Vivado的实时语法检查 (Linter)

仅仅有代码高亮是不够的，我们需要让VS Code在编写代码时就能调用Vivado的编译器进行语法检查，指出错误位置，而不是等到在Vivado中点击Run Synthesis才报错。

以 **Verilog-HDL/SystemVerilog (mshr-h)** 插件为例：

1. 确保已安装该插件。
 2. 在VS Code中打开设置 (`Ctrl + ,`)，搜索 `verilog.linting.linter`。
 3. 将Linter下拉菜单的值修改为 `xvlog`（这是Vivado自带的语法分析器）。
 4. （关键步骤）将Vivado的 `xvlog` 路径添加到你的Windows系统环境变量 `Path` 中。通常路径如下（根据你的Vivado版本自行调整）：
`<你所存放的位置>\Xilinx\Vivado\202X.X\bin`
 5. 配置完成后，重启VS Code。此时，当你在VS Code中保存 `.v` 或 `.sv` 文件时，底部终端会自动调用Vivado的编译器检查语法，并在代码行使用波浪线标出Error或Warning。
-

4. 进阶工作流建议

在上述配置完成后，你可以在VS Code中完成 **编写代码 -> 实时语法检查 -> 保存** 的全过程，只需将Vivado挂在后台，仅用于执行综合（Synthesis）、实现（Implementation）和生成比特流（Generate Bitstream）。

如果你想进一步脱离Vivado图形界面，可以在VS Code的集成终端中使用 `Tcl` 脚本直接调用Vivado进行编译和烧录，这在学术界和企业级开发中十分常见。